

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів

Кафедра систем управління літальних апаратів

Лабораторна робота №6

з дисципліни «Алгоритмізація та програмування»

(назва дисципліни)

на тему: «Реалізація алгоритмів обробки одновимірних масивів мовою C ++»

XAI.301.312.G7.3.ЛР

Виконав: здобувач 1 курсу, групи 312

напряму підготовки (спеціальності)

G7 «Автоматизація комп'ютерно-інтегровані
технології та робототехніка»

ОПП Інженерія мобільних додатків

(шифр і назва напряму підготовки (спеціальності))

С. С. БОЯРСЬКИЙ

(прізвище й ініціали студента)

Прийняв: доцент каф.301, канд. техн. наук,

О. В. ГАВРИЛЕНКО

(посада, науковий ступінь, прізвище й ініціали)

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення одновимірних і масивів на мові C ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль одновимірних масивів на мові C ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести весь масив у рядок в порядку зростання індексів, потім – елементи чи підраховані результати відповідно до завдання. Варіанти завдань представлено в табл.1.

Завдання 2. Вирішити завдання на перетворення одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести у консоль заданий масив, потім – змінений. Варіанти завдань представлено в табл.2.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок

ВИКОНАННЯ РОБОТИ

Завдання 1

Дан масив розміру N. Знайти номери тих елементів масиву, які більше свого лівого сусіда, і кількість таких елементів. Знайдені номери виводити в порядку їх зменшення.

Вхідні дані:

N: Ціле число, константа. Максимально допустимий розмір масиву N = 50).

`n`: Ціле число (`int`). Фактична кількість елементів у масиві. Обмеження: $2 < n < 50$.

`arr`: Масив цілих чисел (`int`) розміром `n`.

Вихідні дані:

Вихідний масив: Елементи масиву в порядку зростання індексів.

`result_index`: Ціле число (`int`). Порядковий номер (починаючи з 1) першого знайденого локального мінімуму. Якщо мінімумів немає — результат 0.

Діаграма активності показана на рис. 1

Скріншоти виконання наведено на рис. 2

Лістинг коду вирішення задач наведено в дод. А (стор. 10).

Алгоритм функції task_array_32 (Більші за лівого сусіда)

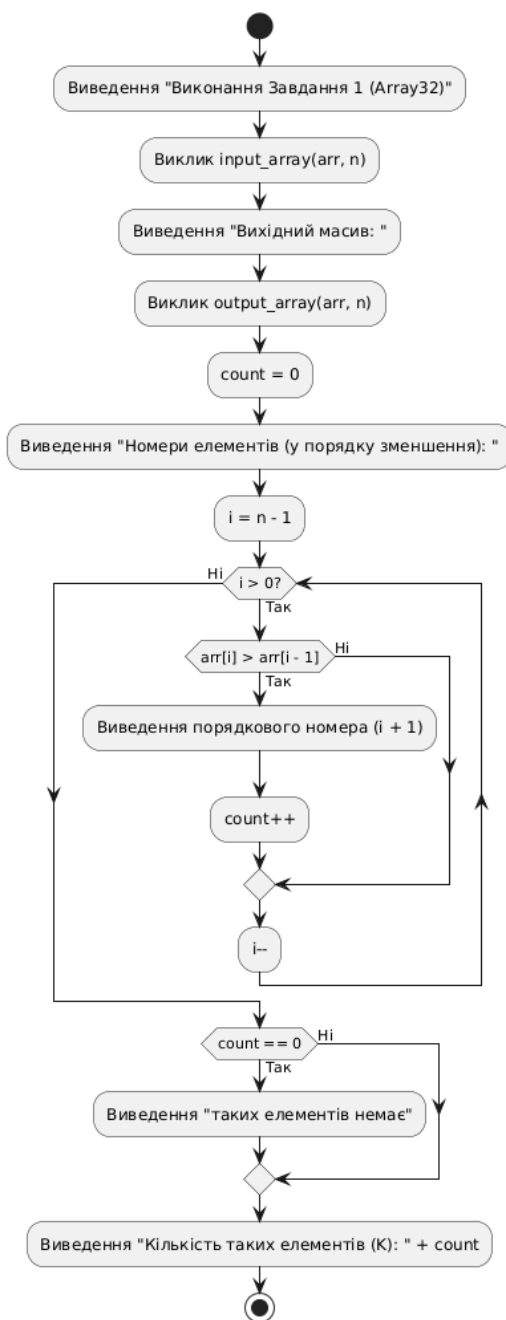


Рисунок 1 – Діаграма активності для алгоритму вирішення задачі 1

```

Виконання Завдання 1 (Array32)
Введіть кількість елементів (2-50): 5
Елемент 1: 5
Елемент 2: 5
Елемент 3: 5
Елемент 4: 5
Елемент 5: 5
Вихідний масив: 5 5 5 5 5
Номери елементів (у порядку зменшення): таких елементів немає
Кількість таких елементів (K): 0

Виконання Завдання 1 (Array32)
Введіть кількість елементів (2-50): 5
Елемент 1: 2
Елемент 2: 5
Елемент 3: 8
Елемент 4: 9
Елемент 5: 100
Вихідний масив: 2 5 8 9 100
Номери елементів (у порядку зменшення): 5 4 3 2
Кількість таких елементів (K): 4

```

Рисунок 2 – Виконання завдання 1

Завдання 2

Дан цілочисельний масив розміру N. Видалити з масиву всі однакові елементи, залишивши їх останні входження.

Вхідні дані

n: Ціле число (int). Кількість елементів у масиві. Обмеження: від 2 до 50

arr: Масив цілих чисел (int).

Вихідні дані:

Масив до змін: Початковий набір даних.

Змінений масив: Результуючий масив, у якому видалені всі однакові елементи, крім їх останнього входження.

Діаграма активності показана на рис. 3

Скріншоти виконання наведено на рис. 4

Лістинг коду вирішення задач наведено в дод. А (стор. 10).

Алгоритм функції task_array_97 (Видалення дублікатів)

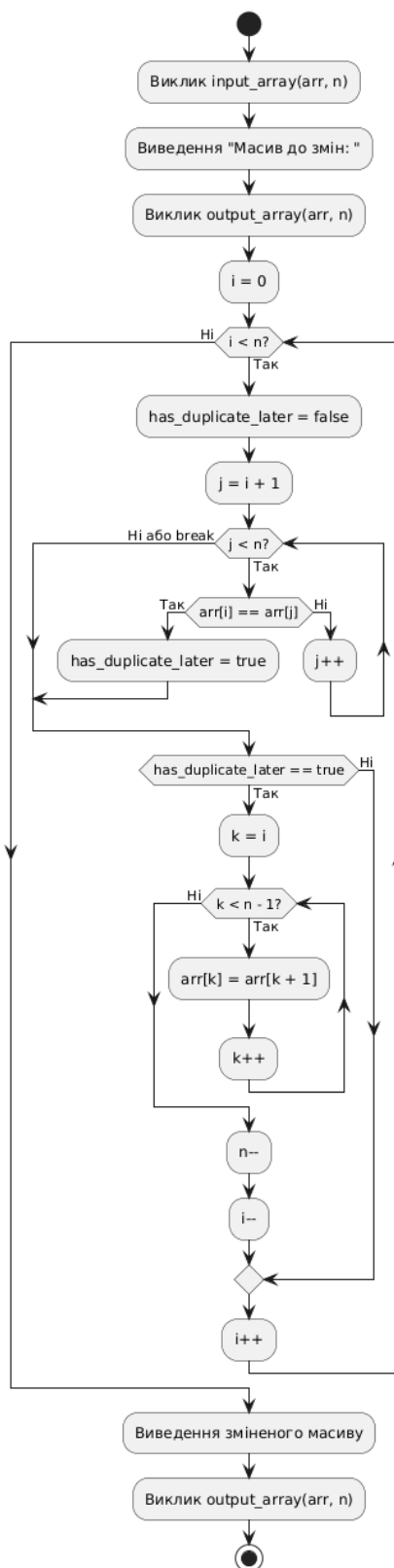


Рисунок 3 – Діаграма активності для алгоритму вирішення задачі 2

```
Введіть кількість елементів (2-50): 5
Елемент 1: 6
Елемент 2: 4
Елемент 3: 8
Елемент 4: 6
Елемент 5: 7
Масив до змін: 6 4 8 6 7
Масив після видалення дублікатів (залишено останні): 4 8 6 7
```

Рисунок 4 – Виконання завдання 2

Завдання 3.

Вхідні дані

choice: Ціле число (int). Номер обраного завдання з меню.

Обмеження: Ціле число від 1 до 2 для вибору задачі або 0 для виходу .

Вихідні дані:

Виклик процедури: Перехід до виконання функцій task_array_32() або task_array_97().

Діаграма активності показана на рис. 5

Скріншоти виконання наведено на рис. 6

Лістинг коду вирішення задач наведено в дод. А (стор. 10).

Алгоритм головної функції main (Меню)

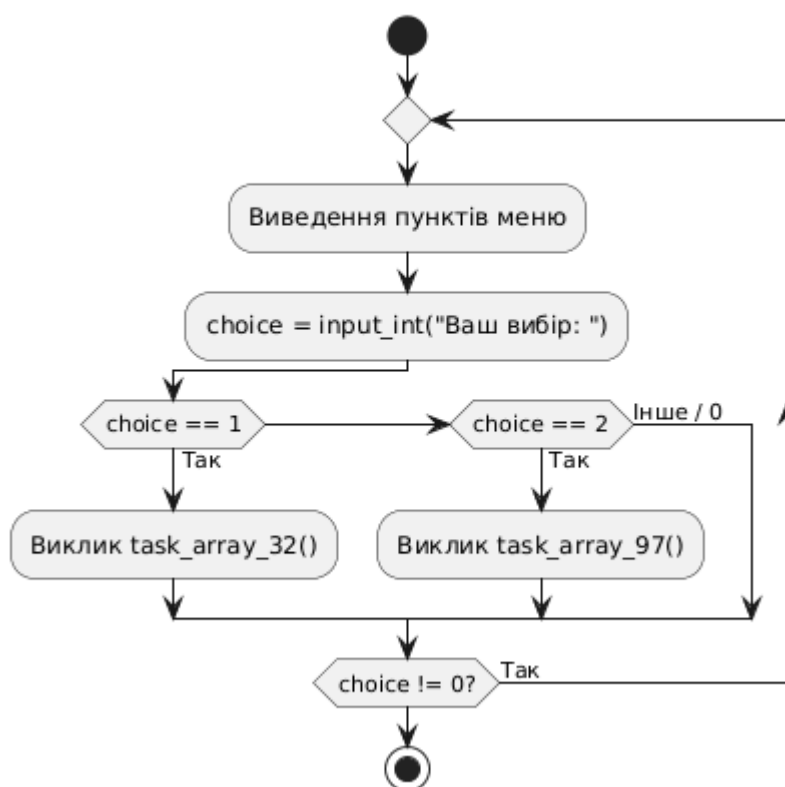


Рисунок 5 – Діаграма активності для алгоритму вирішення задачі 3

```

--- МЕНЮ ---
1. Завдання 1 (Array32)
2. Завдання 2 (Array97)
0. Вихід
Ваш вибір: 2
  
```

Рисунок 6 – Виконання завдання 3

Завдання 3

Діалог у ChatGPT для самоаналізу наведено у дод. В.

ВИСНОВКИ

У ході виконання лабораторної роботи було детально вивчено

теоретичний матеріал щодо основ представлення одновимірних масивів у мові C++. На практиці успішно закріплено навички декларації статичних масивів, а також розроблено власні функції для їх коректного введення з консолі та структурованого виведення результатів.

ДОДАТОК А ЛІСТИНГ КОДУ ПРОГРАМИ

```
#include <iostream>
#include <string>

using namespace std;

// Константа для максимального розміру масиву
const int N = 50;

// Оголошення функцій (прототипи), щоб main міг їх викликати
int input_int(string message);
void input_array(int arr[N], int &n);
void output_array(const int arr[N], int n);
void task_array_32();
void task_array_97();

// Головна функція з меню
int main() {
    int choice;
    do {
        cout << "\n--- МЕНЮ ---" << endl;
        cout << "1. Завдання 1 (Array32)" << endl;
        cout << "2. Завдання 2 (Array97)" << endl;
        cout << "0. Вихід" << endl;
        choice = input_int("Ваш вибір: ");

        if (choice == 1) task_array_32();
        else if (choice == 2) task_array_97();

    } while (choice != 0);

    return 0;
}

// Функція для введення цілого числа (проста версія)
int input_int(string message) {
    int value;
    cout << message;
    cin >> value;
    return value;
}
```

```

// Введення масиву
void input_array(int arr[N], int &n) {
    n = input_int("Введіть кількість елементів (2-50): ");
    if (n > N) n = N;

    for (int i = 0; i < n; i++) {
        arr[i] = input_int("Елемент " + to_string(i + 1) + ": ");
    }
}

// Виведення масиву
void output_array(const int arr[N], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// ЗАВДАННЯ 1: Array32 (Більші за лівого сусіда, зворотний порядок)
void task_array_32() {
    cout << "\nВиконання Завдання 1 (Array32)" << endl;
    int arr[N], n;
    input_array(arr, n);

    cout << "Вихідний масив: ";
    output_array(arr, n);

    int count = 0;
    cout << "Номери елементів (у порядку зменшення): ";

    // Йдемо з кінця масиву до другого елемента (індекс 1)
    // Перший елемент (індекс 0) не має лівого сусіда, тому його не перевіряємо
    for (int i = n - 1; i > 0; i--) {
        if (arr[i] > arr[i - 1]) {
            cout << i + 1 << " "; // Виводимо порядковий номер (індекс + 1)
            count++;
        }
    }

    if (count == 0) {
        cout << "таких елементів немає";
    }

    cout << "\nКількість таких елементів (K): " << count << endl;
}

// ЗАВДАННЯ 2: Array97
// Видалити всі однакові елементи, залишивши їх останні входження
void task_array_97() {
    int arr[N];
    int n;
    input_array(arr, n);

    cout << "Масив до змін: ";
    output_array(arr, n);

    // Йдемо по масиву
    for (int i = 0; i < n; i++) {
        bool has_duplicate_later = false;

```

```

// Перевіряємо, чи є таке ж число далі в масиві
for (int j = i + 1; j < n; j++) {
    if (arr[i] == arr[j]) {
        has_duplicate_later = true;
        break;
    }
}

// Якщо таке число далі є, то поточне (arr[i]) треба видалити
if (has_duplicate_later) {
    for (int k = i; k < n - 1; k++) {
        arr[k] = arr[k + 1];
    }
    n--; // Зменшуємо розмір
    i--; // Перевіряємо цей же індекс знову, бо там тепер нове число
}

cout << "Масив після видалення дублікатів (залишено останні): ";
output_array(arr, n);
}

```

ДОДАТОК В

ДІАЛОГ З ШІ ДЛЯ САМОАНАЛІЗУ

Кілька запитань задай будь ласка мені неважких для самоаналізу

Без проблем. Ось три неважких, але цілком практичних питання. Вони допоможуть тобі переконатися, що ти дійсно розумієш кожний рядок написаного коду перед захистом.

1. **Межі масиву:** У першому завданні ми йдемо по масиву з кінця і перевіряємо умову `arr[i] > arr[i - 1]`. Чому наш цикл зупиняється на `i > 0`, а не йде до `i >= 0`? Що сталося б, якби ми спробували виконати перевірку для нульового індексу?
2. **Структура програми:** Викладач вимагав поставити `int main()` на самий початок файлу. Завдяки чому компілятор C++ не видає помилку і розуміє, що функція `task_array_32()` існує, хоча її детальний код написаний аж у кінці файлу?
3. **Логічне видалення:** У другому завданні, коли ми видаляємо дублікат, ми зсуваємо елементи вліво і робимо `n--`. Чому ми фізично не видаляємо комірку з пам'яті комп'ютера, а просто зменшуємо значення змінної `n`?

Спробуєш відповісти на ці питання своїми словами прямо зараз?

1. тому що ми вказуємо `i` мінус 1

2. Ми використовуємо прототип за допомогою `void`, щоб визначити, що така функція буде нижче

3. Тому що ми і так даємо зрозуміти, що робочих елементів буде тепер на один менше

Ти вловив суть.

1. **Абсолютно правильно.** Саме через перевірку $i - 1$ ми не можемо перевіряти нульовий індекс, щоб не вийти за межі масиву і не зламати програму.
2. **Майже ідеально.** Головне слово тут дійсно — **прототип**. Для викладача варто лише уточнити, що `void` — це тип повертаемого значення (який вказує, що функція нічого не повертає), а сам прототип — це весь рядок `void task_array_32()`; Але загальну механіку ти описав вірно: ми анонсуємо компілятору, що реалізація буде нижче.
3. **У саме яблучко.** Це технічно називається «логічне видалення» або «зміна логічного розміру». Фізично комірки залишаються, але програма просто ігнорує їхнє існування під час подальшої роботи.